

■■ Hoja de Referencia Rápida: Spring Session

■ 1. Anotaciones de Habilitación de Proveedores de Sesión

Anotación	Respaldo	Caso de Uso
<code>@EnableJdbcHttpSession</code>	Base de Datos Relacional (SQL).	Aplicaciones relacionales sin Redis.
<code>@EnableRedisHttpSession</code>	Memoria en caché (Redis).	Microservicios de alta concurrencia.
<code>@EnableMongoHttpSession</code>	Base de datos NoSQL (MongoDB).	Entornos NoSQL nativos.
<code>@EnableSpringHttpSession</code>	Almacén personalizado.	Repositorio a la medida.

■■ 2. Configuración de Spring Session JDBC (`application.yml`)

```
spring:
  datasource:
    url: jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1
    username: sa
    password:
  session:
    store-type: jdbc
    jdbc:
      initialize-schema: always
```

```
table-name: SPRING_SESSION
timeout: 1800s
```

```
@Configuration
@EnableJdbcHttpSession(maxInactiveIntervalInSeconds = 1800)
public class HttpSessionConfig {

    @Bean
    public CookieSerializer cookieSerializer() {
        DefaultCookieSerializer serializer =
            new DefaultCookieSerializer();
        serializer.setCookieName("JSESSIONID");
        serializer.setCookiePath("/");
        serializer.setUseHttpOnlyCookie(true);
        serializer.setSameSite("Lax");
        return serializer;
    }
}
```

■ 3. Inyección y Manipulación de Sesión en Controladores Web

```
@Controller
@RequestMapping("/carrito")
public class CarritoController {

    @GetMapping
    public String verCarrito(
        @SessionAttribute(name = "usuario", required = false)
        UsuarioDTO usuario,
        HttpSession session,
        Model model) {
        if (usuario == null) {
            return "redirect:/login";
        }

        CarritoDTO carrito =
            (CarritoDTO) session.getAttribute("CARRITO");
        if (carrito == null) {
```

```

        carrito = new CarritoDTO();
        session.setAttribute("CARRITO", carrito);
    }

    model.addAttribute("carrito", carrito);
    return "carrito";
}

@GetMapping("/logout")
public String cerrarSesion(HttpSession session) {
    session.invalidate();
    return "redirect:/login?logout";
}
}

```

■ 4. Inyección del Repositorio de Sesiones (SessionRepository)

```

@Service
public class AdministracionSesionesService {

    private final FindByNameSessionRepository<? extends Session>
        sessionRepository;

    public AdministracionSesionesService(
        FindByNameSessionRepository<? extends Session> repo) {
        this.sessionRepository = repo;
    }

    public Map<String, ? extends Session> obtenerSesiones(
        String username) {
        return sessionRepository.findByPrincipalName(username);
    }

    public void revocarSesion(String sessionId) {
        sessionRepository.deleteById(sessionId);
    }
}

```

■ 5. Transferencia de Sesión: Cookies vs Encabezados HTTP

```
@Bean
public HttpSessionIdResolver httpSessionIdResolver() {
    // Usa el header X-Auth-Token en lugar de cookies
    return HeaderHttpSessionIdResolver.xAuthToken();
}
```